



The obstacle that we wish to overcome

- Last week, you learned that a particle filter seeks to evaluate:

$$f(x_k | \mathbb{Z}_{k-1}) = \int_{R_{x_{k-1}}} f(x_k | x_{k-1}) f(x_{k-1} | \mathbb{Z}_{k-1}) dx_{k-1}$$

$$f(x_k | \mathbb{Z}_k) = \frac{f(z_k | x_k) f(x_k | \mathbb{Z}_{k-1})}{f(z_k | \mathbb{Z}_{k-1})}.$$

- You also learned how to approximate the integrals in these expressions using a brute-force numeric approach.
- However, the numeric approach requires $\mathcal{O}(N_r^{2n})$ function evaluations;
 - Finding the optimal solution via numeric integration is usually not practical since it requires operation over a high-dimensional space.
- We need to find a way to beat the “curse of dimensionality” and get good approximate results using less computation.



Idea

- Suppose that the integral to solve can be factored as:

$$\mu = \int_{R_x} g(x) f(x) dx,$$

where $f(x)$ has all of the properties of a pdf (x can be multi-dimensional).

- Then, we recognize that $\mu = \mathbb{E}[g(x)]$ if we treat x as an RV.
- Key idea: Write the integral as an expectation, then approximate the expectation with a sample mean based on sampling from the known pdf $f(x)$.
- That is, suppose we draw $N \gg 1$ random samples from pdf $f(x)$. Then,

$$\mu = \int_{R_x} g(x) f(x) dx = \mathbb{E}[g(x)] \approx \frac{1}{N} \sum_{i=1}^N g(x^{(i)}),$$

where $x^{(i)}$ is the i th sample drawn from $f(x)$, not x to the power i .



Monte–Carlo integration

- This idea of approximating an integral by recasting the problem as a sample mean—which is then evaluated by drawing random samples from a pdf—is known as “Monte–Carlo integration.”
 - The brute-force numeric approach that we looked at last week evaluates the integrand on a regular grid over the entire range R_x of the integration variable x ,
 - But, the Monte–Carlo method *randomly* chooses points from R_x at which the integrand is evaluated—no need to sample with uniform spacing.
 - Moreover, with likelihood proportional to the significance of that region of R_x .
 - As we don’t need to sample the integrand on a regular grid, we can *significantly* trim the number of function evaluations required to approximate an integral.
- Formal result: If the samples are independent, then the estimate is unbiased and will “almost surely” converge to μ as $N \rightarrow \infty$.
- Practical result: It can do well for finite N also.



Example of Monte–Carlo integration

- Suppose we are given a random process $y_k = \cos(x_k)$ where $x_k \sim \mathcal{N}(0, \sigma_x^2)$ and we wish to estimate:

$$\mathbb{E}[y] = \int_{-\infty}^{\infty} \cos(x) \frac{1}{\sqrt{2\pi\sigma_x^2}} \exp\left(\frac{-x^2}{2\sigma_x^2}\right) dx.$$

- Comparing to our prior form, $\mu = \int g(x) f(x) dx$, we assign:

$$g(x) = \cos(x) \quad \text{and} \quad f(x) = \frac{1}{\sqrt{2\pi\sigma_x^2}} \exp\left(\frac{-x^2}{2\sigma_x^2}\right).$$

- So, we will draw N independent Gaussian RVs $x^{(i)}$, each having zero mean and standard deviation σ_x , and compute:

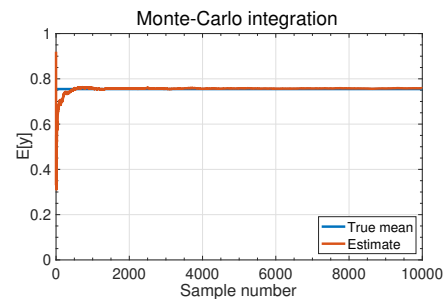
$$\mathbb{E}[y] \approx \frac{1}{N} \sum_{i=1}^N \cos(x^{(i)}).$$



Octave code for example

```
N = 10000; sx = 0.75; % problem constants
x = sx*randn(1,N); % generate sample pts.
y = cos(x); % function evaluations
mu = cumsum(y)./(1:N); % running approx.
plot(1:N,mu); % plot estimate vs N
```

- Figure shows one trial run of the above code.
 - Results will be somewhat different from one trial run to another due to the randomness inherent to the Monte–Carlo method.
 - The “true mean” can be found analytically to be $\exp(-\sigma_x^2/2)$, or found numerically.
- We see quite fast convergence to the neighborhood of the solution.



Summary

- We wish to approximate a high-dimensional integral efficiently.
- If the integrand can be factored into the product of a function $g(x)$ and a pdf $f(x)$, then Monte–Carlo integration draws random samples $x^{(i)}$ from $f(x)$ and approximates the integral as the sample mean of $g(x^{(i)})$.
 - This avoids sampling the integrand over a full regular grid, where some sample values are relatively unimportant.
- Can *significantly* reduce the level of computation required to approximate an integral, especially for high-dimensional problems.